Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE1113 Sistemas Empotrados
Prof. Dr.-Ing. Jeferson González Gómez

Taller 9: SoC con Nios V/m en Platform Designer

Del diseño del SoC a la verificación en simulación RTL (sin tarjeta física)

1. Introducción

Los talleres anteriores trabajaron el software empotrado sobre procesadores *dados*: firmware bare-metal y un RTOS sobre un RISC-V emulado en QEMU (Talleres 06–08). En este taller se da el paso complementario: **construir el procesador y su sistema** —un sistema en chip (SoC)— sobre una FPGA, usando el procesador **Nios V/m** de Altera, un *soft core* RISC-V que se instancia dentro de la lógica programable. Este es exactamente el tipo de sistema sobre el que se desarrolla el Proyecto 2 del curso (co-diseño hardware/software para la tarjeta DE1-SoC), por lo que este taller sirve de preparación directa.

Nios V es el sucesor de Nios II (el *soft core* propio de Altera, de ISA privativa) y, al ser **RISC-V**, comparte la misma arquitectura que el firmware visto en los Talleres 06–08. El cambio de Nios II a Nios V trae consigo un cambio de herramientas de software: desaparecen el Nios II EDS y su IDE, y en su lugar se usan las utilidades `niosv-*` (con el toolchain `riscv32-unknown-elf`) y, opcionalmente, el IDE RiscFree. El taller incluye *scripts* que automatizan el flujo de forma reproducible y versionable con Git, en el espíritu de los `scripts/build.sh` del Taller 06; de todas formas, el SoC puede construirse igualmente desde la interfaz gráfica de Platform Designer. Use la vía que le resulte más cómoda: lo importante es el sistema que se diseña, no la herramienta con que se edita.

Un punto importante: a diferencia de Nios II, **Nios V no tiene un simulador de conjunto de instrucciones (ISS)**. Para ejecutar el software sin la tarjeta física, se simula el **RTL del SoC completo** en Questa, cargando en la RAM on-chip la imagen del programa. Es más pesado que un ISS, pero es el flujo oficial y permite observar, además del software, el comportamiento del hardware que se diseña (como los PIO que se agregan en la parte evaluada).

2. Conceptos fundamentales

2.1. Platform Designer y el SoC

Platform Designer (antes Qsys) es la herramienta de Altera para ensamblar un SoC a partir de bloques de propiedad intelectual (IP): procesadores, memorias y periféricos se instancian y se interconectan mediante el bus **Avalon** (memory-mapped), sin escribir a mano la lógica de interconexión. El sistema se describe en un archivo `.qsys`; al generarlo, Platform Designer produce el RTL sintetizable, un archivo `.sopcinfo` (descripción del sistema que consume el generador de BSP) y, opcionalmente, un *testbench* de simulación.

2.2. Nios V/m

Nios V/m es la variante *microcontrolador* de Nios V: un núcleo RISC-V RV32I compacto, sin MMU, pensado para control y tareas de firmware. Expone, entre otras, interfaces Avalon de *host* para instrucciones y datos (`instruction_manager`, `data_manager`), una interfaz de reloj, una de reset, un receptor de interrupciones (`platform_irq_rx`) y un módulo de depuración (`dm_agent`). A diferencia de Nios II, no tiene un vector de excepción configurable en el hardware: en RISC-V la base de trampas (`mtvec`) la fija el software en el arranque. Solo hay que indicarle de qué memoria arranca (`resetSlave`).

2.3. Las herramientas del flujo

El SoC puede diseñarse y generarse tanto desde la interfaz gráfica de Platform Designer como desde la línea de comandos. Los *scripts* del taller usan las utilidades de consola que acompañan a Quartus, que conviene conocer (todas tienen su equivalente en la GUI):

Herramienta	**Rol**
`qsys-script`	Crea/edita el `.qsys` con comandos Tcl (alternativa a la GUI).
`qsys-generate`	Genera el RTL de síntesis, el `.sopcinfo` y el testbench.
`niosv-bsp`	Genera el *Board Support Package* (HAL) a partir del `.sopcinfo`.
`niosv-app`	Genera el proyecto CMake de la aplicación enlazado contra el BSP.
`cmake`/`make`	Compilan y enlazan el `hello.elf`.
`elf2hex`	Convierte el ELF en `ram.hex`, imagen con que se inicializa la RAM on-chip (lo invoca el *make* automáticamente).
`vsim` (Questa)	Simula el RTL del SoC; el `printf` aparece en el *transcript*.

2.4. E/S mapeada a memoria y acceso por puntero

En un SoC Avalon, cada periférico ocupa un rango de direcciones; sus registros se leen y escriben como si fueran memoria. Un PIO (*Parallel I/O*) expone su **registro de datos en el desplazamiento 0** de su base: en un PIO de salida, escribir ese registro fija el valor de los pines de salida; en uno de entrada, leerlo devuelve el valor presente en los pines de entrada.
El BSP genera `system.h` con una macro de dirección base por periférico (por ejemplo `PIO_OUT_BASE`). La forma más directa de acceder a un registro es un **puntero volátil** a esa dirección:

```c
#include "system.h"
#include <stdint.h>

volatile uint32_t *pio_out = (volatile uint32_t *)PIO_OUT_BASE;
volatile uint32_t *pio_in  = (volatile uint32_t *)PIO_IN_BASE;

*pio_out = 0xA5;                /* escribe el registro de datos (
   offset 0) */
uint32_t v = *pio_in;           /* lee el registro de datos (offset
   0)     */
```

El calificador `volatile` es obligatorio: le prohíbe al compilador eliminar o reordenar el acceso (el valor puede cambiar por fuera del programa, y la escritura tiene un efecto en el hardware aunque nunca se vuelva a leer). En la parte evaluada se exige este estilo, **sin** las funciones del driver HAL (`alt_*`) ni las macros `IOWR`/`IORD`, para practicar la E/S mapeada a memoria "a mano", tal como lo pide el Proyecto 2.

3. El SoC base que se entrega

El archivo `hw/hello.qsys` contiene un SoC mínimo pero completo, ya validado en simulación. Se puede abrir en la GUI de Platform Designer o regenerar desde cero con `qsys-script -script=hw/build_system.tcl`. Sus componentes y su mapa de direcciones son:

Instancia	IP	Base	Rol
`clk`	`clock_source`	—	Reloj 50 MHz + reset
`cpu`	`intel_niosv_m`	—	Procesador RISC-V
`ram`	`altera_avalon_onchip_memory2`	`0x00000000`	RAM 256 KB (código+datos)
`juart`	`altera_avalon_jtag_uart`	`0x00040000`	`stdout`
`cpu.timer_sw_agent`	(interno)	`0x00040040`	Temporizador del sistema
`cpu.dm_agent`	(interno)	`0x00050000`	Depuración (alineado a 64 KB)

El procesador arranca desde la RAM (`resetSlave = ram.s1`, `resetOffset = 0`). El BSP detecta automáticamente que la JTAG UART es el `stdout` y que el temporizador del sistema es el del propio Nios V. El reloj y el reset se exportan al tope del sistema; en simulación los estimula el *testbench* generado (no hay que escribir uno a mano).

Dos detalles no obvios descubiertos al construir este SoC, útiles al agregar periféricos:

- El `dm_agent` (módulo de depuración) exige una base **alineada a 64 KB** (p. ej. `0x40000` o `0x50000`).
- Las interrupciones en Nios V se conectan con el receptor (`platform_irq_rx`) como *inicio* de la conexión, al revés que en Nios II.

4. Requisitos e instalación

Este taller usa herramientas de FPGA que se instalan de forma nativa (no hay contenedor Docker como en los talleres 06–08). Se requiere:

- **Quartus Prime Lite 25.1std** (gratuita; soporta Cyclone V, el dispositivo de la DE1-SoC, y Nios V).
- El componente **Nios V / RiscFree**, que aporta el toolchain `riscv32-unknown-elf` y las utilidades `niosv-*`.
- **Questa Altera Starter FPGA Edition** (incluida con Quartus).

La Starter Edition de Questa es gratuita pero requiere una licencia *node-locked*. Un detalle que suele trabar la instalación en Linux: Questa usa licenciamiento **SALT** (Siemens), de modo que la variable de entorno es `SALT_LICENSE_SERVER` (apuntando al archivo de licencia), **no** `LM_LICENSE_FILE`.

4.1. Configuración del entorno

El script `scripts/env.sh` deja listas todas las variables; basta con `source scripts/env.sh`. Lo que hace, por si hay que ajustarlo a otra instalación, es: fijar `ALTERA_ROOT` a la raíz de

la instalación (p. ej. `~/altera_lite/25.1std`) y, a partir de ella, exportar las rutas de las herramientas y la licencia:

```
export QUARTUS_ROOTDIR="$ALTERA_ROOT/quartus"
export PATH="$QUARTUS_ROOTDIR/bin:$QUARTUS_ROOTDIR/sopc_builder/bin:\
$ALTERA_ROOT/niosv/bin:\
$ALTERA_ROOT/riscfree/toolchain/riscv32-unknown-elf/bin:\
$ALTERA_ROOT/questa_fse/bin:$PATH"
# Questa usa SALT, no FLEXlm:
export SALT_LICENSE_SERVER="$HOME/.altera.quartus/questa_lic.dat"
```

Cada ruta del `PATH` aporta: `quartus/bin` (`quartus_sh`, `quartus_pgm`), `quartus/sopc_builder/bin` (`qsys-generate`, `qsys-edit`, `ip-catalog`), `niosv/bin` (`niosv-bsp`, `niosv-app`, `elf2hex`), el toolchain `riscv32-unknown-elf` y `questa_fse/bin` (`vsim`).
La licencia Starter es *node-locked* a la MAC del equipo. Para confirmar que la suya corresponde, compare la MAC con el `HOSTID` del archivo, y al final verifique que las herramientas responden:

```
cat /sys/class/net/*/address                # MACs del equipo
grep -o 'HOSTID=[0-9a-f]*' "$SALT_LICENSE_SERVER"   # debe coincidir
   con una
riscv32-unknown-elf-gcc --version           # toolchain RISC-V
qsys-generate --help                        # Platform Designer
vsim -version                               # Questa
```

Si se usa `LM_LICENSE_FILE` en lugar de `SALT_LICENSE_SERVER`, Questa falla con `Invalid license environment. Application closing.`

5. Inicio rápido

Todo el flujo se corre desde el directorio del taller. Primero se prepara el entorno (ajuste `ALTERA_ROOT` si su Quartus está en otra ruta):

```
source scripts/env.sh
```

Luego, el hardware y el software:

```
scripts/build-hw.sh    # qsys-generate: RTL + .sopcinfo + testbench
scripts/build-sw.sh    # niosv-bsp + niosv-app + cmake/make -> hello.
   elf, ram.hex
```

El paso de hardware (`build-hw.sh`) equivale a abrir `hw/hello.qsys` en Platform Designer y pulsar *Generate HDL + Generate Testbench System*; si prefiere la GUI, hágalo así y continúe con `build-sw.sh`.
Y la simulación en Questa, en modo consola:

```
scripts/sim.sh
```

En el *transcript* debe aparecer la salida del Nios V:

```
# Hola mundo, aqui el Nios V/m reportandose 0...
# ...
# Fin del programa.
```

Para capturar un *waveform* (útil para evidenciar los PIO), use la GUI:

```
scripts/sim.sh gui
#  en la consola de Questa:
#     add wave -r /*
#     run 3 ms
```

6. Parte 1: reproducir el hola mundo

Antes de modificar nada, reproduzca el flujo completo con el SoC base y el programa `sw/src/hello_world.c` tal como se entrega, y confirme que el procesador arranca y ejecuta el `printf`. Esta parte verifica que su instalación (Quartus, Nios V, Questa y la licencia SALT) está correcta.

Actividades

1. Corra `build-hw.sh`, `build-sw.sh` y `sim.sh`, y capture el *transcript* con el "hola mundo".
2. Abra `build/bsp/system.h` e identifique las bases de `ram`, `juart` y el temporizador. Relacione esas direcciones con el mapa de la Sección 3.

7. Parte 2 (evaluada): dos PIO conectados por software

El objetivo es ampliar el SoC con **dos PIO** y manejarlos por software con acceso directo por puntero. El trabajo tiene tres frentes: hardware, software y verificación.

7.1. Hardware: agregar los PIO

Agregue al sistema dos instancias de `altera_avalon_pio` ("PIO (Parallel I/O)" en el catálogo), por ejemplo:

- `pio_out`: ancho 8 bits, dirección **salida** ("output"), pensado como 8 LED.
- `pio_in`: ancho 4 bits, dirección **entrada** ("input"), pensado como 4 interruptores.

Para cada PIO: conecte su reloj a `clk.clk`, su reset a `clk.clk_reset`, y su puerto Avalon al `cpu.data_manager`; **exporte** el conducto externo (los pines) al tope del sistema; y asigne una base que no choque con las existentes (p. ej. `0x00040010` y `0x00040020`). Lo más cómodo es hacerlo en la **interfaz gráfica de Platform Designer** (`qsys-edit hw/hello.qsys`): arrastrar los IP, conectarlos en la matriz, exportar sus conductos y dejar que la herramienta asigne las bases. Es la vía recomendada y no se penaliza usarla. Como alternativa (o para dejar el sistema reproducible), el mismo cambio puede escribirse en `hw/build_system.tcl`, antes de `save_system`, con líneas como:

```
add_instance pio_out altera_avalon_pio
set_instance_parameter_value pio_out {direction} {Output}
set_instance_parameter_value pio_out {width} {8}
add_connection clk.clk pio_out.clk
add_connection clk.clk_reset pio_out.reset
add_connection cpu.data_manager pio_out.s1
set_connection_parameter_value cpu.data_manager/pio_out.s1 baseAddress
    0x00040010
# ... (analogo para pio_in con direction=Input, width=4) y exportar
   conductos
```

Tras modificar el sistema, **regenere todo**: `build-hw.sh` y luego `build-sw.sh` (el BSP se regenera y `system.h` pasa a definir `PIO_OUT_BASE` y `PIO_IN_BASE`).

7.2. Software: acceso por puntero directo

En `sw/src/hello_world.c`, después del "hola mundo", agregue el acceso a los dos PIO usando **punteros volátiles** a sus bases (de `system.h`), como se mostró en la Sección 2.4. El programa debe, como mínimo:

- escribir al menos tres valores distintos en `pio_out` (p. ej. `0x01`, `0xA5`, `0xFF`);
- leer `pio_in` e imprimir su valor por `printf`.

Restricción: el acceso a los PIO debe hacerse con dereferencia de punteros `volatile`; **no** se permiten las macros del driver (`IOWR_*/IORD_*`) ni funciones `alt_*`. (El `printf` de la consola sí puede usarse: es la salida estándar, no un acceso al PIO.)

7.3. Verificación en simulación

Como no hay tarjeta, los pines de los PIO se observan y estimulan en Questa:

- **Salida (`pio_out`):** en la GUI (`sim.sh gui`), agregue al *waveform* el conducto exportado del `pio_out` y corra la simulación; debe ver cómo cambia a los valores que escribió el software.
- **Entrada (`pio_in`):** el puerto de entrada está sin manejar en el *testbench*; fuércelo desde la consola de Questa antes de que el software lo lea, por ejemplo:

```
force -freeze sim:/hello_tb/.../pio_in_export 4'hA␣0
```

  (use `find signals sim:/hello_tb/*` para ubicar la ruta exacta del puerto). Confirme que el valor que imprime el software coincide con el que forzó.

Opcionalmente, para una prueba auto-contenida sin `force`, puede conectar en el *testbench* generado el conducto de `pio_out` al de `pio_in` (*loopback*) y verificar que el software lee de vuelta lo que escribió; documente esta variante si la usa.

Entregable de evidencia

Una **captura de pantalla** que muestre, en una misma ejecución, el "hola mundo" en el *transcript* y la evidencia del funcionamiento de los PIO (el *waveform* del `pio_out` cambiando y/o el valor de `pio_in` impreso por el software coincidiendo con el forzado).

8. Evaluación

El taller se evalúa sobre la Parte 2 (la Parte 1 es prerrequisito de verificación del entorno) y el informe.

Hardware: los PIO en el SoC (30 %)

R1. Las dos instancias de PIO (una de salida, una de entrada) agregadas, conectadas (reloj, reset, Avalon) y exportadas, con bases sin conflicto (construidas en la GUI o por Tcl, es indiferente).
R2. El sistema regenera sin errores (`build-hw.sh`) y `system.h` define las bases de ambos PIO.

Software: acceso por puntero (35 %)

R3. Escritura de al menos tres valores a `pio_out` y lectura de `pio_in`, mediante punteros `volatile` a las bases de `system.h`.
R4. **Sin** usar macros `IOWR`/`IORD` ni funciones `alt_*` para el acceso a los PIO.

Verificación (20 %)

R5. Captura de pantalla del "hola mundo" y de los PIO en funcionamiento (waveform del `pio_out` y/o lectura de `pio_in` impresa coincidiendo con el valor forzado).

Informe (15 %)

R6. Informe PDF (máximo 3 páginas) con: el mapa de direcciones final del SoC, el fragmento de código del acceso por puntero, y la captura de evidencia con una breve explicación de qué muestra.

Entregable

- Archivo `ApellidoNombre_Taller09.tar.gz` con:
  1. `hw/hello.qsys` (o `build_system.tcl`) con los PIO.
  2. `sw/src/hello_world.c` modificado.
  3. `README.txt`: nombre, carné, bases asignadas a los PIO e instrucciones de reproducción.
- Informe PDF y la captura de evidencia.

Referencias

[1] Altera (Intel), *Nios V Embedded Processor Design Handbook* [en línea], disponible en: `https://www.intel.com/content/www/us/en/docs/programmable/726952/`.

[2] Altera (Intel), *Nios V Processor Software Developer Handbook* [en línea], disponible en: `https://www.intel.com/content/www/us/en/docs/programmable/743810/`.

[3] Altera (Intel), *Quartus Prime Pro Edition User Guide: Platform Designer* (UG-20130) [en línea].

[4] Altera (Intel), *Embedded Peripherals IP User Guide* (UG-01085), capítulo PIO Core [en línea].